

Prompt Engineering Reference Guide

practical patterns for working with AI models

Prompting is the main lever we have for getting good, consistent output out of an LLM, and yet most people treat it as an afterthought. This guide pulls together the prompting patterns that actually show up in day-to-day work, why they matter, and where they tend to go wrong. It's written for the team, not for a textbook, so expect opinions and shortcuts alongside the theory.

A quick note before you dive in: none of these techniques are magic. They're just ways of giving the model better constraints and better signal. The model still has to know the domain. Good prompting narrows the gap between what you meant and what you typed; it doesn't replace expertise.

Table of Contents

Introduction

Core Prompting Techniques

[Zero-Shot Prompting](#)
[One-Shot Prompting](#)
[Few-Shot Prompting](#)
[Role Prompting](#)
[Persona Prompting](#)
[Instruction Prompting](#)
[Context-Based Prompting](#)
[Chain-of-Thought Prompting](#)
[Step-by-Step Prompting](#)
[Structured Output Prompting](#)
[Template Prompting](#)
[Constraint-Based Prompting](#)
[Multi-Turn Prompting](#)
[Prompt Chaining](#)
[Self-Refinement Prompting](#)

Other Practical Techniques Worth Knowing

[System Prompting](#)
[Analogical Prompting](#)
[Negative Prompting](#)
[Rephrase-and-Respond](#)

Comparison Table

Prompt Writing Best Practices

Common Prompting Mistakes

Reusable Prompt Template Library

Introduction

Every prompting technique below is really just answering one of three questions for the model: what do you want, how should you get there, and what does “done” look like. Zero-shot and instruction prompting are mostly about the “what.” Few-shot, chain-of-thought, and step-by-step are about the “how.” Structured output and constraint-based prompting are about “done.”

Once you see it that way, picking a technique stops being guesswork. You look at what's going wrong in your output and match it to the gap: vague answers need clearer instructions, inconsistent formatting needs structured output prompting, and wrong reasoning needs a technique that slows the model down before it commits to an answer.

Most real prompts combine two or three of these techniques at once. A good production prompt might have a role, a set of constraints, an example, and a structured output format all in the same message. Treat the list below as building blocks, not a menu where you pick one.

Core Prompting Techniques

Zero-Shot Prompting

You ask the model to do a task with no examples, just an instruction. It relies entirely on what the model already learned during training to figure out the pattern.

When to use	Common, well-understood tasks: summarizing, translating, classifying sentiment, answering factual questions, basic rewriting. Anything the model has almost certainly seen thousands of times before.
Best practices	Be explicit about the output you want. Don't assume the model will infer format or tone from nothing. State the task, the audience, and any hard limits (length, format) up front.
Advantages	Fast to write, cheap to run (fewer tokens), and works well for straightforward tasks. No need to hunt for good examples.
Common mistakes	Using it for tasks with a specific house style or unusual format, then being surprised the output doesn't match. Also, being too vague (“summarize this”) and getting a generic result back.

Simple example

Summarize the following paragraph in two sentences: [paragraph text]

Real-world example

Classify the following support ticket as Billing, Technical, or Account Access. Respond with only the category name. Ticket: “I was charged twice for my subscription this month and need a refund for the duplicate charge.”

One-Shot Prompting

Same idea as zero-shot, but you give the model exactly one example of the input/output pattern before asking it to do the real task. It's the minimum amount of demonstration you can give.

When to use	When the task is simple but has one quirk the model won't guess on its own, like a specific tone, a naming convention, or an unusual output shape.
Best practices	Make the example genuinely representative of the real task, not an edge case. Keep it short so it doesn't eat your context budget for no reason.
Advantages	Cheaper than few-shot, and often enough to lock in format or tone when zero-shot is too loose.
Common mistakes	Picking an example that's too different from the actual input, which teaches the model the wrong pattern. Also, forgetting that one example can't cover variation, so if your real inputs vary a lot, one-shot won't be enough.

Simple example

Convert the sentence into a formal tone. Example: "hey can u send me the file" → "Could you please send me the file?" Now convert: "yo did we ever fix that bug"

Real-world example

Rewrite each product feature into a customer-facing benefit statement, matching this style. Example: "Supports SSO" → "Log in once and access everything, no extra passwords to manage." Now rewrite: "Includes automated daily backups"

Few-Shot Prompting

You give the model multiple examples (usually 2 to 5) of the input/output pattern before the real request. This is one of the most reliable ways to steer format, tone, and reasoning style without fine-tuning anything.

When to use	Tasks with a specific structure, classification schemes with several categories, extraction tasks with a particular output shape, or anytime zero-shot output is inconsistent across runs.
Best practices	Use varied examples that cover the range of inputs you expect, including at least one edge case. Keep formatting between examples identical so the pattern is unambiguous. Order matters less than people think, but put the most representative example last, closest to the real task.
Advantages	Much more consistent output than zero-shot. Great for locking in a JSON schema, a scoring rubric, or a tone that's hard to describe in words but easy to show.
Common mistakes	Overloading the prompt with 10+ examples when 3 good ones would do, wasting context and sometimes confusing the model with redundant patterns. Also using examples that are all near-identical, which doesn't teach the model how to handle variation.

Simple example

Extract the sentiment as Positive, Negative, or Neutral. “The delivery was late but the product works great.” → Neutral “Absolutely thrilled with this purchase.” → Positive “Terrible packaging, item arrived broken.” → Negative “Customer service resolved my issue within minutes.” →

Real-world example

Given a customer email, extract the intent as JSON with fields: category, urgency (low/medium/high), and requires_response (true/false). [Three examples of emails with their JSON output] Now process: “Hi, just wanted to say the new update looks great, no issues on my end.”

Role Prompting

You assign the model a professional role or area of expertise, like “You are a senior security auditor,” before giving the task. This nudges the model to pull from the vocabulary, priorities, and depth associated with that role.

When to use	Tasks that benefit from domain framing, like code review, legal-style caveats, financial analysis, or technical writing aimed at a specific audience.
Best practices	Be specific about the role and its scope: “senior backend engineer reviewing for security issues” beats “expert.” Pair the role with the actual task instructions rather than relying on the role alone to carry the whole prompt.
Advantages	Shifts vocabulary and focus without needing to write out a long list of instructions. Also helps set the right level of formality and depth.
Common mistakes	Treating the role as a substitute for real constraints. Saying “you are a doctor” doesn’t make medical advice accurate, it just changes tone. Also stacking too many roles at once (“act as a doctor, lawyer, and marketer”), which produces muddled output.

Simple example

You are an experienced copyeditor. Review this paragraph for grammar and clarity issues only, don’t change the meaning: [paragraph]

Real-world example

You are a senior DevOps engineer reviewing a Terraform script for a production AWS environment. Flag any security misconfigurations, missing tags, or resources without least-privilege IAM policies. [script pasted below]

Persona Prompting

Related to role prompting but focused on voice and character rather than expertise. You’re asking the model to write as someone, with a distinct tone, vocabulary, and personality, not just to apply domain knowledge.

When to use	Marketing copy, social content, brand voice work, creative writing, or any case where tone and personality matter more than technical accuracy.
Best practices	Describe the persona with concrete traits (word choice, sentence length, energy level) rather than abstract adjectives like “fun” or “professional,” which the model interprets loosely. Give a short sample of the voice if you have one.
Advantages	Produces distinctive, less generic-sounding copy. Useful for keeping brand voice consistent across many pieces of content.
Common mistakes	Confusing persona with role and expecting a “witty pirate” persona to also give accurate technical advice. Also over-specifying to the point the output feels like a caricature.

Simple example

Write a product announcement in the voice of a laid-back, slightly sarcastic tech reviewer. Keep sentences short.

Real-world example

Write our weekly newsletter intro in the voice of our brand: direct, a little irreverent, no corporate jargon, short punchy sentences, occasional self-deprecating humor. Topic: we just shipped a much-requested dark mode.

Instruction Prompting

A direct, imperative-style prompt that tells the model exactly what to do, step by step if needed, without examples or role framing. It's the plainest form of prompting, closest to giving someone a checklist.

When to use	Tasks where the requirements are precise and well understood, like data transformations, formatting rules, or multi-part requests where you need every part addressed.
Best practices	Use numbered or bulleted instructions when there are multiple requirements. Put constraints and the output format at the end, since models tend to weight the last few lines heavily. Avoid burying the actual ask in a wall of context.
Advantages	Very predictable, easy to debug (if it fails, you know exactly which instruction wasn't followed), and doesn't require crafting examples.
Common mistakes	Writing overly long lists of instructions that contradict each other, or that pile on so many requirements the model starts dropping some. Also being ambiguous about the order of operations when order matters.

Simple example

Rewrite the email below. Remove all exclamation points, shorten it to under 100 words, and end with a clear call to action.

Real-world example

Take the meeting transcript below and: 1) list the decisions made, 2) list open action items with owners if mentioned, 3) flag any decision that contradicts an earlier one in the transcript. Use headers for each section.

Context-Based Prompting

You provide relevant background information (documents, data, prior conversation, company-specific facts) alongside the instruction so the model isn't answering from general knowledge alone. This is the backbone of most retrieval-augmented workflows.

When to use	Anytime the answer depends on information the model wouldn't otherwise know: your company's policies, a specific document, recent data, or facts unique to your situation.
Best practices	Put the context before the instruction so the model reads it first, or clearly label where context ends and the task begins. Trim irrelevant material, since dumping in an entire document when only one section matters increases the odds the model latches onto the wrong part. Tell the model explicitly to rely only on the provided context if that matters for accuracy.
Advantages	Grounds the response in facts you control instead of the model's training data, which reduces made-up details and outdated information.
Common mistakes	Providing too much irrelevant context and expecting the model to find the needle in the haystack reliably. Also forgetting to tell the model what to do if the context doesn't contain the answer, which invites guessing.

Simple example

Using only the policy text below, answer the question: does our return policy cover items purchased more than 90 days ago? [policy text]

Real-world example

Here is our Q2 product roadmap doc and last month's customer feedback summary. Based only on this material, draft three talking points for tomorrow's stakeholder update on why feature X was deprioritized.

Chain-of-Thought Prompting

You prompt the model to work through a problem in a structured, logical progression before landing on a final answer, rather than jumping straight to a conclusion. This is especially useful for anything involving multi-step logic, math, or decisions with several dependent factors.

When to use	Math problems, multi-step logic puzzles, decisions that depend on weighing several factors, or any task where jumping to a snap answer tends to produce mistakes.
--------------------	---

Best practices	Ask the model to work through the problem systematically and explain its reasoning before giving the final answer. Where possible, separate the reasoning from the final answer so downstream systems can grab just the answer (for example, “end your response with Final Answer: ...”). For high-stakes tasks, ask the model to double-check its own logic before finalizing.
Advantages	Meaningfully improves accuracy on tasks with several interdependent steps, and gives you a way to spot where the logic went wrong if the final answer is off.
Common mistakes	Using it for simple lookups or classification tasks where it just adds latency and cost for no benefit. Also, treating the visible reasoning as proof of correctness. Models can produce reasoning that sounds coherent but doesn't actually reflect why they landed on their answer, so always verify the conclusion independently for anything important.

Simple example

A store has 84 apples. They sell 27 in the morning and receive a delivery of 40 more in the afternoon, then sell 15 more. Work through this step by step and give the final count of apples remaining.

Real-world example

We're deciding between two vendors for our data warehouse migration based on cost, migration timeline, and team familiarity with the platform. Walk through the tradeoffs for each factor before recommending one. [vendor details below]

Step-by-Step Prompting

Similar spirit to chain-of-thought, but focused specifically on breaking a task into an ordered sequence of actions rather than reasoning through a judgment call. Think procedures and workflows, not logic puzzles.

When to use	Tasks with a clear procedural structure: writing setup instructions, walking through a debugging process, generating a checklist, or explaining how to configure something.
Best practices	Ask explicitly for numbered steps. If some steps are conditional (“if X, do Y instead”), say so directly rather than expecting the model to infer branching logic.
Advantages	Produces output that's immediately usable as documentation or a runbook. Easier for a human to follow and verify than a single dense paragraph.
Common mistakes	Asking for “step by step” on a task that isn't actually sequential, which forces the model to invent an artificial order. Also not specifying the level of granularity, leading to steps that are either too vague or absurdly over-detailed.

Simple example

Explain, step by step, how to set up a Python virtual environment on macOS.

Real-world example

Write a step-by-step runbook for rotating an expired SSL certificate on our production load balancer, including the verification step to confirm the new cert is live before closing the ticket.

Structured Output Prompting

You explicitly tell the model to return output in a specific format, like JSON, XML, Markdown tables, or YAML, usually with a defined schema. This matters most when the output feeds into another system rather than being read directly by a person.

When to use	Any time the output needs to be parsed programmatically, inserted into a database, rendered in a UI, or kept consistent across many runs (like extraction pipelines or automated reports).
Best practices	Show the exact schema you want, including field names and types, ideally with a short example. State what to do with missing fields (omit, use null, use empty string) so the model doesn't guess inconsistently. If you need strictly parseable output, ask for the format with no extra commentary before or after it.
Advantages	Massively more reliable to parse downstream, easier to validate, and removes ambiguity about what a “good” response looks like.
Common mistakes	Asking for JSON but not specifying the schema, resulting in inconsistent field names between runs. Also forgetting that models sometimes wrap JSON in explanatory text or markdown code fences even when told not to, so downstream code should handle that gracefully rather than assuming a perfectly clean response.

Simple example

Extract the name, email, and phone number from this text and return it as JSON with keys “name”, “email”, “phone”. Use null for any missing field.

Real-world example

Parse the job posting below and return a JSON object with: title, company, location, salary_range (or null), required_skills (array), and years_experience (number or null). Return only the JSON, no other text.

Template Prompting

You build a reusable prompt “shape” with placeholders that get swapped out for different inputs, rather than writing a fresh prompt every time. This is how most production prompting actually works once a task gets repeated more than a couple of times.

When to use	Recurring tasks with the same structure but different inputs, like generating product descriptions, writing status update summaries, or running the same QA check across many documents.
--------------------	--

Best practices	Keep the fixed instructions and formatting rules in the template, and isolate only the truly variable parts as placeholders. Test the template against a few edge-case inputs, not just the typical case, before rolling it out. Version your templates so you can track what changed when output quality shifts.
Advantages	Consistency at scale, faster iteration (you tweak the template once instead of every individual prompt), and easier to hand off to teammates who don't need to reinvent the wording each time.
Common mistakes	Making a template too rigid for the range of inputs it actually needs to handle, so it breaks or produces awkward output on anything slightly unusual. Also not revisiting templates after the underlying model or task changes.

Simple example

Write a one-paragraph product description for {{product_name}}, a {{product_category}}, highlighting {{key_benefit}}. Tone: {{tone}}.

Real-world example

Weekly status update template used across engineering teams: “Summarize this week's work for the {{team_name}} team based on the ticket list below. Structure as: Shipped, In Progress, Blocked, Next Week. Flag anything blocked for more than {{threshold_days}} days with a warning icon.”

Constraint-Based Prompting

You define hard boundaries on the output, length limits, forbidden words or topics, required elements, tone restrictions, so the model has less room to wander. Constraints work best stacked with other techniques rather than used alone.

When to use	Anytime there's a hard requirement that absolutely must be respected: character limits for a UI field, compliance language that must appear, or content that must be excluded for legal or brand reasons.
Best practices	State constraints as explicit rules, not soft suggestions (“keep it brief” is weaker than “maximum 50 words”). List multiple constraints as a short bulleted list rather than burying them mid-paragraph. For anything truly critical, verify the output against the constraint programmatically rather than trusting the model got it exactly right.
Advantages	Reduces the need for manual editing afterward and keeps output usable in downstream systems with strict format or length requirements.
Common mistakes	Piling on so many constraints that they conflict with each other (asking for a “comprehensive” answer in “one sentence”). Also assuming a numeric constraint like word count will be hit exactly. Models approximate length; treat limits as targets to verify, not guarantees.

Simple example

Write a product tagline. Maximum 8 words. No exclamation points. Must include the word “simple.”

Real-world example

Write the meta description for this landing page. Hard limit: 155 characters. Must include the primary keyword “cloud backup software.” Do not use superlatives like “best” or “#1” for legal review reasons.

Multi-Turn Prompting

Rather than trying to get everything right in a single message, you work with the model across a back-and-forth conversation, refining, correcting, and adding detail turn by turn. This mirrors how people actually work with AI assistants day to day.

When to use	Exploratory work, iterative drafting, debugging sessions, or anything where you don't know the full requirements upfront and want to shape the output as you go.
Best practices	Give specific, actionable feedback between turns (“make the second paragraph shorter and cut the jargon” beats “make it better”). Reference earlier parts of the conversation explicitly if the thread is long, since models can lose track of which version of something you're referring to. Periodically summarize where things stand if the conversation gets long, to keep everyone (including the model) aligned.
Advantages	Lets you course-correct cheaply instead of trying to perfectly specify everything in one shot. Well suited to creative or ambiguous work where the target only becomes clear as you iterate.
Common mistakes	Letting a conversation run so long that earlier context gets crowded out or contradicted, causing the model to lose the thread. Also giving vague feedback like “not quite right” without saying what specifically is wrong.

Simple example

Turn 1: “Draft a tweet announcing our new feature.” Turn 2: “Good start, but cut the hashtags and make the first sentence punchier.”

Real-world example

Turn 1: “Help me outline a technical blog post about our new caching layer.” Turn 2: “The outline's good, but move the benchmarks section earlier and add a section comparing it to Redis.” Turn 3: “Now write the introduction based on this outline, roughly 150 words.”

Prompt Chaining

You break a complex task into a sequence of separate prompts, where the output of one becomes the input to the next. Unlike multi-turn prompting, which is conversational and often manual, chaining is usually a deliberate pipeline, sometimes fully automated.

When to use	Complex, multi-stage tasks where doing everything in one prompt would overload the model or make output hard to verify at each stage, like research-then-write, extract-then-summarize-then-format, or draft-then-critique-then-revise pipelines.
Best practices	Keep each step in the chain focused on one clear job. Validate or sanity-check the output of each step before feeding it into the next, especially in automated pipelines, since errors compound down the chain. Design steps so a person could audit any single link without needing the whole chain's context.
Advantages	Each step is simpler and more reliable than trying to cram the whole task into one prompt. Easier to debug (you can isolate which link in the chain is producing bad output) and easier to swap out or improve individual steps later.
Common mistakes	Chaining too many steps for a task that a single well-written prompt could handle, adding latency and cost with no quality benefit. Also not handling failure at intermediate steps, so a bad output early in the chain silently poisons everything downstream.

Simple example

Step 1: "Extract the key data points from this earnings report." Step 2 (using Step 1's output): "Turn these data points into a two-paragraph summary for a non-financial audience."

Real-world example

Step 1: Summarize each of 20 customer interview transcripts into key pain points. Step 2: Cluster the pain points from all 20 summaries into common themes. Step 3: Turn the themes into a prioritized list with supporting quotes, formatted for a product team readout.

Self-Refinement Prompting

You ask the model to review and improve its own output, either by critiquing a first draft against specific criteria or by comparing multiple attempts and picking the strongest. It's essentially asking the model to be its own first-pass editor.

When to use	Writing tasks where quality matters more than speed, code that needs a second look for bugs or edge cases, or any output where a fresh critique pass tends to catch issues the first pass missed.
Best practices	Give the model concrete criteria to critique against (clarity, accuracy, tone, adherence to constraints) rather than a vague "check your work." Separate the critique step from the revision step for better results: first ask what's wrong, then ask for a revised version based on that critique. Don't loop this indefinitely; two passes usually capture most of the benefit, and returns drop off fast after that.
Advantages	Catches obvious errors, awkward phrasing, or missed constraints without needing a human review pass first. Often produces a noticeably tighter final draft.

Common mistakes

Treating repeated self-refinement as a guarantee of correctness. It doesn't verify facts against reality, it just polishes based on the model's own judgment, so factual errors can survive multiple "refined" passes. Also asking for refinement without specific criteria, which produces cosmetic changes rather than real improvement.

Simple example

Here's a draft email. First list three things that could be clearer or more concise. Then rewrite the email addressing those points.

Real-world example

Review this SQL query for correctness and performance issues, specifically N+1 patterns and missing indexes implied by the WHERE clauses. List the issues first, then provide a revised query.

Other Practical Techniques Worth Knowing

These come up less often in day-to-day prompting but are worth having in your back pocket.

System Prompting

A system prompt sets standing rules for an entire session rather than a single message, things like tone, scope, safety boundaries, or output format that should apply no matter what the user asks next. It's distinct from role prompting in that it's usually set once, up front, and persists, rather than being restated per task. Use it when you're building an assistant or tool that needs consistent behavior across many different user requests, not just a one-off task.

Example: "You are a customer support assistant for a SaaS billing product. Only answer questions related to billing, invoices, and subscriptions. If asked about anything else, politely redirect. Always respond in under 100 words."

Analogical Prompting

You ask the model to draw on a similar, better-understood problem or example before tackling the actual one. This helps with tasks that benefit from pattern-matching against a known structure, like explaining a new concept in terms of a familiar one, or approaching an unfamiliar problem by first recalling how a similar problem was solved.

Example: "Before designing this caching strategy, think about how a library's checkout system prevents two people from borrowing the same physical book. Apply a similar principle to prevent two processes from writing to the same cache key."

Negative Prompting

You explicitly tell the model what to avoid, rather than only stating what to include. This works well as a supplement to instructions, not a replacement, since models generally respond better to being told what to do than what not to do.

Example: "Write a project update. Do not use the phrases 'circle back,' 'synergy,' or 'move the needle.' Do not include a call to action at the end."

Rephrase-and-Respond

You ask the model to first restate or clarify the question or task in its own words before answering it. This surfaces ambiguity early and often catches misunderstandings before they show up in a wrong answer, especially useful for vague or underspecified requests.

Example: “Before answering, restate what you understand the question to be asking. Then answer it. Question: ‘How do we fix the churn problem?’”

Comparison Table

A quick-reference summary of everything above, side by side.

Technique	Ideal Use Case	Strengths	Limitations	Example Scenario
Zero-shot	Simple, common tasks	Fast, cheap, no setup	Inconsistent on niche or nuanced tasks	Summarizing a news article
One-shot	Simple task with one specific quirk	Cheap way to lock in format/tone	Doesn't cover varied inputs	Rewriting a sentence to match one tone example
Few-shot	Tasks needing consistent structure or classification	High consistency, teaches pattern by example	Uses more tokens, needs good examples	Extracting structured data from varied emails
Role prompting	Domain-specific framing	Shifts vocabulary/depth quickly	Doesn't guarantee factual accuracy	Code review as a “senior engineer”
Persona prompting	Voice- and tone-driven content	Distinctive, on-brand output	Can feel gimmicky if overused	Brand-voice social copy
Instruction prompting	Well-defined, precise tasks	Predictable, easy to debug	Can get unwieldy with too many rules	Multi-part data reformatting
Context-based prompting	Answers depending on specific info	Grounded, reduces made-up facts	Needs relevant, well-scoped context	Answering from a company policy doc
Chain-of-thought	Multi-step logic or decisions	Better accuracy on complex reasoning	Slower, costs more tokens, not proof of correctness	Comparing vendors across multiple factors
Step-by-step	Procedural, sequential tasks	Clear, actionable output	Forces artificial order on non-sequential tasks	Writing a setup or troubleshooting guide
Structured output	Output feeding other systems	Reliable to parse, consistent schema	Models may add stray text around it	Extracting fields into JSON
Template prompting	Recurring tasks at scale	Consistency, faster iteration	Can be too rigid for edge cases	Weekly status report generation

Technique	Ideal Use Case	Strengths	Limitations	Example Scenario
Constraint-based	Hard requirements (length, banned words)	Reduces manual cleanup	Conflicting constraints confuse the model	Meta description under 155 characters
Multi-turn	Exploratory or iterative work	Cheap course correction	Long threads can lose context	Iterating on a blog post outline
Prompt chaining	Complex, multi-stage pipelines	Simpler steps, easier debugging	Errors compound across steps	Interview transcripts to themed insights report
Self-refinement	Quality-sensitive writing or code	Catches errors and awkward phrasing	No guarantee of factual correctness	Reviewing a SQL query for issues

Prompt Writing Best Practices

Put the important stuff where the model will actually weigh it. Models tend to pay more attention to instructions near the end of a prompt, so if there's context that needs to come first (a document, a transcript, background data), put your actual task and constraints after it, not buried in the middle.

Be specific about the output, not just the task. “Write a summary” and “write a three-sentence summary aimed at someone who's never used the product” produce very different results. The more precisely you describe the shape of a good answer, the less you'll need to fix afterward.

Say what “good” looks like before you ask for the work, not after you've seen a bad draft. If you know you'll want a specific tone, length, or format, put it in the first prompt. It's cheaper than a round of corrections.

Separate instructions from content clearly. Use delimiters, headers, or explicit labels (“Here is the transcript:” ... “Here is your task:”) so the model doesn't confuse pasted content with instructions, especially with longer prompts.

Match the technique to the failure mode you're actually seeing. If output is inconsistent, add examples. If it's vague, tighten the instructions. If the reasoning is wrong, add a technique that slows the model down before the final answer. Don't reach for the most complex technique by default; a lot of tasks are genuinely zero-shot problems.

Iterate like you would with any other tool. Keep a working version, test against a handful of realistic and edge-case inputs, adjust, and retest. A prompt that works on the first try you thought of is rarely as robust as it feels.

Watch your context budget. Long few-shot examples, huge context dumps, and multi-turn threads all compete for the same space. Trim what isn't earning its place in the prompt.

Common Prompting Mistakes

Being vague and hoping the model fills in the gaps sensibly. It often will, but not the way you wanted, and you won't find out until you're staring at the output.

Assuming the model remembers something from an earlier session or a different conversation. Unless it's explicitly in the current context, it isn't there.

Over-stuffing a single prompt with too many unrelated asks. If a prompt is trying to do five different jobs, break it into a chain instead of hoping one giant instruction list holds together.

Treating confident-sounding output as correct output. Fluency isn't accuracy. Anything factual, numerical, or consequential needs a verification step, whether that's a second pass, a human check, or a programmatic validation.

Skipping examples on tasks that clearly need them, then blaming the model for inconsistent formatting. If you can't describe the exact output shape in one clean sentence, show it instead.

Not testing against edge cases. A prompt that works beautifully on the happy-path example often breaks on the weird real-world input that shows up in week two.

Forgetting that constraints are targets, not guarantees, especially numeric ones like word counts or character limits. Verify anything that has to be exact.

Letting long conversations drift without a check-in. If a multi-turn thread has gone on for a while, a quick recap keeps both sides aligned before it goes further off track.

Reusable Prompt Template Library

A set of starting templates for common team tasks. Swap in the bracketed placeholders and adjust constraints as needed.

Writing

Write a [content type] about [topic] for [audience]. Tone: [tone]. Length: [length]. Include: [required elements]. Avoid: [things to exclude].

Coding

Write a [language] function that [does X]. Input: [describe input]. Output: [describe output]. Handle these edge cases: [list]. Include brief comments explaining non-obvious logic. Do not add error handling beyond [scope], keep it minimal.

Testing

Write [unit/integration] tests for the following function using [framework]. Cover the happy path, at least two edge cases, and one failure case. Use descriptive test names. [paste function]

Debugging

Here's the error message and the relevant code. Explain what's causing the error, then propose a fix. If there are multiple possible causes, list them in order of likelihood. Error: [error]. Code: [code].

Documentation

Write developer documentation for the following [function/API endpoint/module]. Include: a one-line summary, parameters with types, return value, one usage example, and any exceptions or edge cases worth calling out. [paste code or spec]

Brainstorming

Generate [number] distinct ideas for [goal]. Constraints: [any limits, budget, audience, format]. For each idea, give a one-line description and one potential downside. Avoid overly similar variations of the same idea.

Summarization

Summarize the following [document/transcript/thread] in [length]. Focus on [specific angle, e.g., decisions made, risks, action items]. Skip [things to exclude, e.g., small talk, tangents]. [paste content]

Translation

Translate the following text into [target language]. Preserve the tone and any technical terminology. Flag any phrase that doesn't have a direct equivalent and explain the choice made. [paste text]

Analysis

Analyze the following [data/report/situation] for [specific question or angle]. Structure your response as: key findings, supporting evidence, and any caveats or limitations in the data. [paste content]

Have a technique that's worked well for you that isn't covered here? Add it to this doc rather than keeping it in your own notes. This is meant to grow as the team's practices evolve.